Matrix Factorization with applications in single-cell RNA-seq

Yongjin Park

05 February 2026

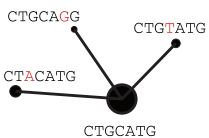
Roadmap of STAT/BIOF/GSAT540 2026

data generation followed by *inference* or validation

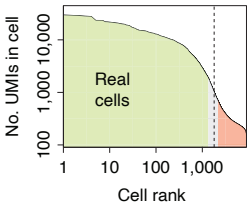
3. Formulating biological questions
4. Hypothesis testing
5. RNA-seq data
6. Inference focusing on RNA-seq
7. Experimental design
8. Single-cell genomics
9. **Matrix factorization** (Today)
10. Genome-wide association studies
11. Multiple hypothesis testing

Recap: Overview of single-cell data analysis

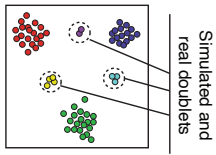
Alignment and molecular counting



Cell filtering and quality control



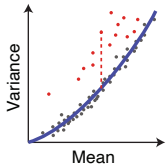
Doublet scoring



Cell size estimation

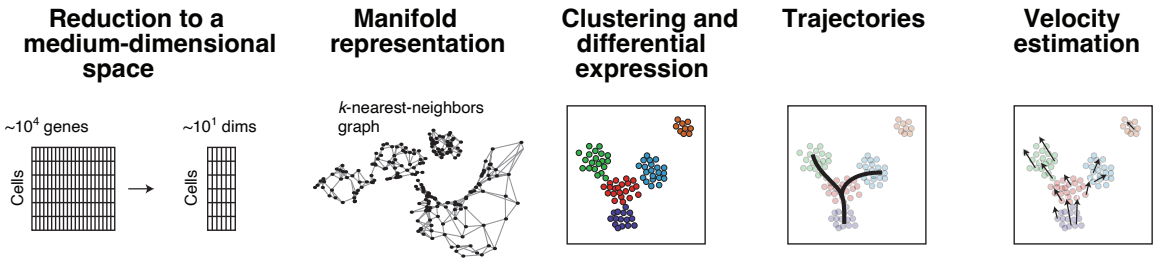
Cells	c_1	c_2	c_3
$Gene_1$	2	4	20
$Gene_2$	1	2	10
$Gene_3$	3	6	30
Cell depth: 6		12	60

Gene variance analysis



(Kharchenko 2021)

Overview of single-cell data analysis cont'd



(Kharchenko 2021)

Learning objectives

- Learn Principal Component Analysis (PCA) and Singular Value Decomposition (SVD)
- Understand matrix factorization and its applications in single-cell analysis
- Introduction to useful tools based on matrix factorization
  - `glmpca`, `rliger`, `fastTopic`, `MOFA`, ...

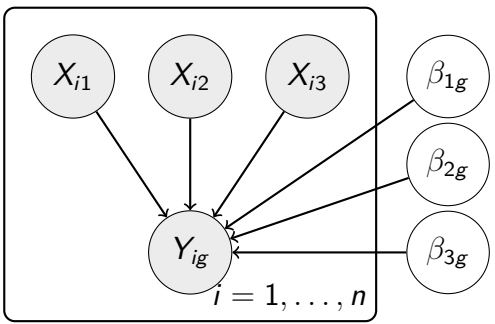
Today's lecture

1. What is Principal Component Analysis?
2. Non-negative Matrix Factorization
3. How can we use factorization results?

Let's start with multivariate linear regression

Linear model for gene g:

$$Y_{ig} = \beta_{1g} \cdot X_{i1} + \beta_{2g} \cdot X_{i2} + \beta_{3g} \cdot X_{i3} + \epsilon_{ig},\ \epsilon_{ig} \sim N(0, \sigma_g^2)$$



- X_{ij}: covariate j for sample i
- β_{jg}: effect of covariate j on gene g
- Y_{ig}: expr. of gene g in sample i

::: ::::::

A design matrix = a column space view

We can rewrite the linear model as:

$$\mathbf{y}_g = \beta_{0g}\mathbf{x}_0 + \beta_{1g}\mathbf{x}_1 + \beta_{2g}\mathbf{x}_2 + \beta_{3g}\mathbf{x}_3 + \boldsymbol{\epsilon}_g$$

where $\mathbf{x}_j$ is the j-th column of $\mathbf{X}$

A design matrix = a column space view

We can rewrite the linear model as:

$$\mathbf{y}_g = \beta_{0g}\mathbf{x}_0 + \beta_{1g}\mathbf{x}_1 + \beta_{2g}\mathbf{x}_2 + \beta_{3g}\mathbf{x}_3 + \boldsymbol{\epsilon}_g$$

where $\mathbf{x}_j$ is the j-th column of $\mathbf{X}$

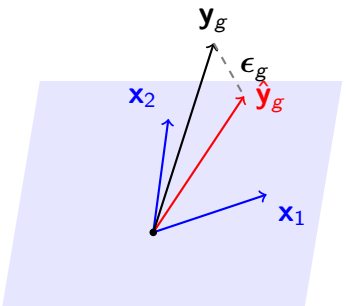
What is a linear regression?

Finding the "best" **linear combination** of column vectors to approximate $\mathbf{y}_g$

- Each column $\mathbf{x}_j$ represents a covariate pattern across all samples
- The coefficients β_{jg} tell us how much each pattern contributes
- The fitted values $\hat{\mathbf{y}}_g = \mathbf{X}\hat{\boldsymbol{\beta}}_g$ lie in the **column space** of $\mathbf{X}$

Linear regression = projection onto linear space

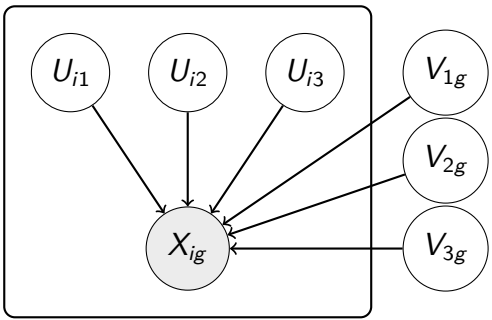
If the whole prediction space can be made up by a linear combination of two n vectors $\mathbf{x}_1$ and $\mathbf{x}_2$...



Column space of $\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2]$

$\hat{\mathbf{y}}_g$ is the **projection** of $\mathbf{y}_g$ onto the column space of $\mathbf{X}$

Linear regression = projection onto some space

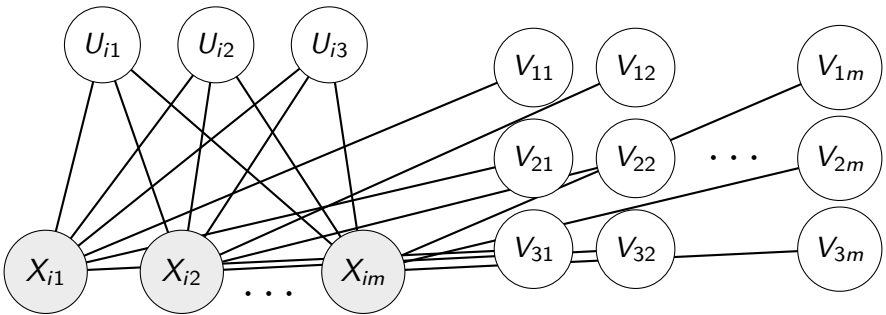


What if our covariates are unknown?

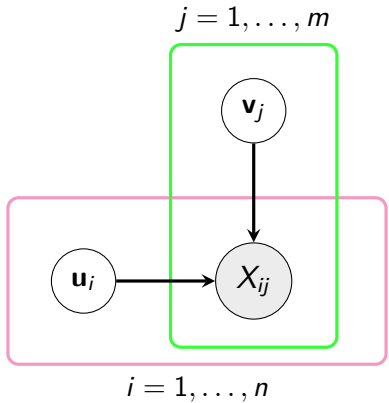
$$Y_{ig} = U_{i1}V_{1g} + U_{i2}V_{2g} + X_{i3}V_{3g} + \epsilon_{ig}$$

- U_{ij}: **latent** factor j for sample i
- V_{jg}: loading of gene g on factor j
- Y_{ig}: observed expression of gene g in sample i

Many linear regression models on many genes/columns



Many linear regression models on many genes/columns

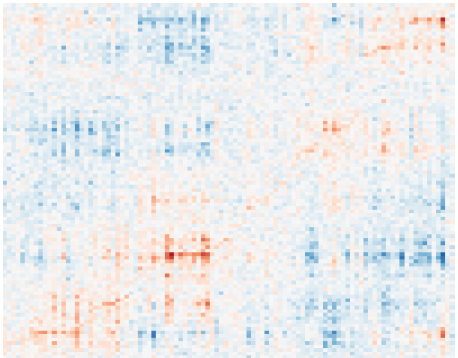


The goal is to find hidden patterns

A data matrix was generated by the following:

$$\mathbf{x}_i = \mathbf{u}_i V^\top + \epsilon_i$$

for all i with $\epsilon_i \sim \mathcal{N}(\mathbf{0}, I)$



Can we reverse engineer the U and V matrices from the data X?

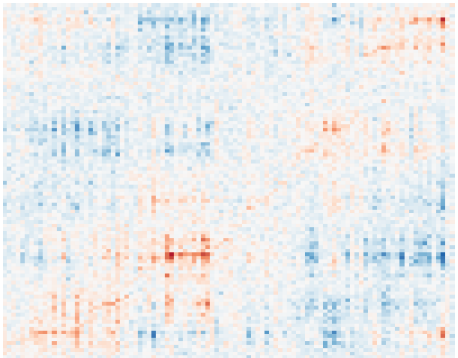
$$X \to UV^\top$$

The goal is to find hidden patterns

A data matrix was generated by the following:

$$\mathbf{x}_i = \mathbf{u}_i V^\top + \epsilon_i$$

for all i with $\epsilon_i \sim \mathcal{N}(\mathbf{0}, I)$



Can we reverse engineer the U and V matrices from the data X?

$$X \to UV^\top$$

features

X

feature loading

Let's look at the PDAC 10x Genomics data

```
PDAC_TISSUE_1/
|-- barcodes.tsv.gz    (9 KB)
|-- features.tsv.gz    (303 KB)
+-- matrix.mtx.gz      (17 MB)
```

- **barcodes.tsv.gz** - Cell IDs
- **features.tsv.gz** - Genes
- **matrix.mtx.gz** - Counts (sparse)

Matrix Market format

- Sparse matrix standard
- Memory efficient
- Compatible with Seurat, Scanpy

Loading 10x data with Seurat (R)

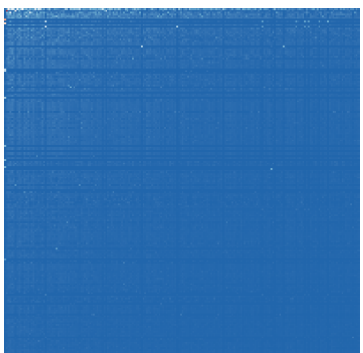

```
## An object of class Seurat
## 19273 features across 1410 samples within 1 assay
## Active assay: RNA (19273 features, 0 variable features)
##  1 layer present: counts
```

Extract the count matrix for later use:

```
## [1] 19260  1410
```

Let's just use top 200 most variable genes and cells

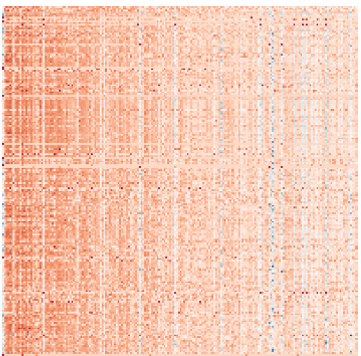
Top 200 genes



- We will call such a high-dimensional matrix X $(m \times n)$
- X has m=19,260 rows (transcripts/genes/features)
- X has n=1,410 columns (cells/#data points)
- The rows were log-transformed and scaled by `scale()` for visualization
- Each cell is a 19,260-dimensional vector!
- Each gene is a 1,410-dimensional vector...

Let's just use top 200 most variable genes and cells

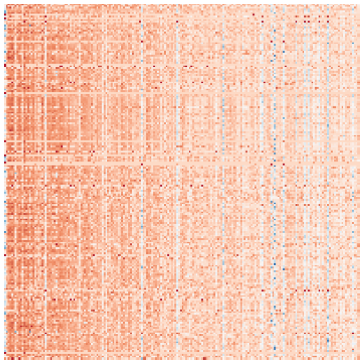
Top 200 genes



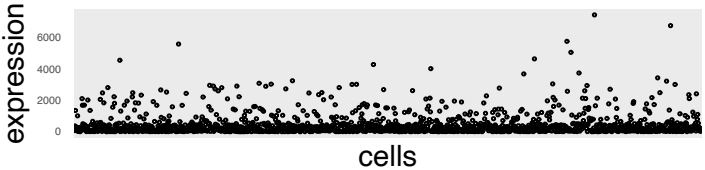
- We will call such a high-dimensional matrix X ($m \times n$)
- X has m=19,260 rows (transcripts/genes/features)
- X has n=1,410 columns (cells/#data points)
- The rows were log-transformed and scaled by `scale()` for visualization
- Each cell is a 19,260-dimensional vector!
- Each gene is a 1,410-dimensional vector...

1-dimensional representations/summary of data matrix

Top 200 genes

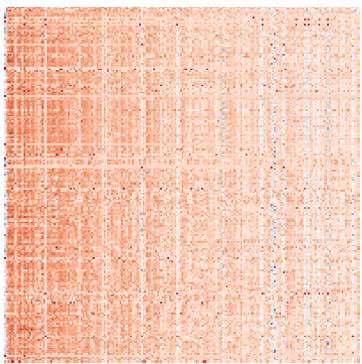


Most variable cell (column):

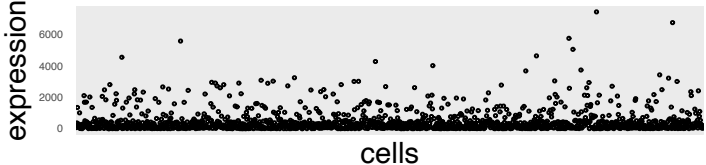


1-dimensional representations/summary of data matrix

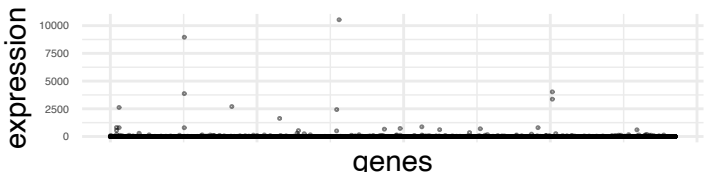
Top 200 genes



Most variable cell (column):

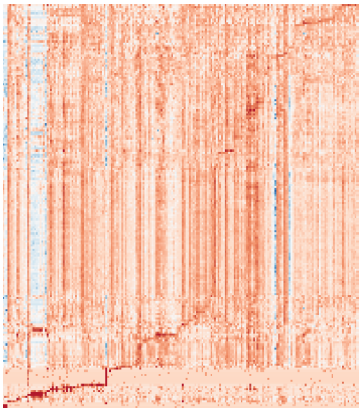


Most variable gene (row):



Can you see some common patterns?

200 genes and cells



Common patterns will become more apparent after applying PCA.

How do we recover “common” patterns from data?

Principal Component Analysis

(Pearson 1901)

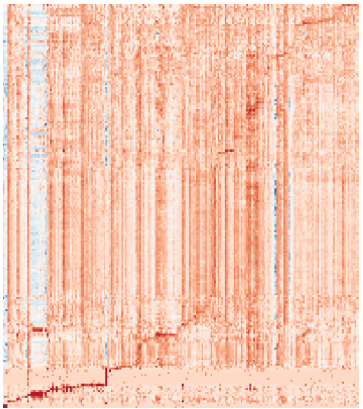
- Projection [of the original data] that minimizes the projection cost between the original and projected
- The cost = mean squared distance

(Hotelling 1933)

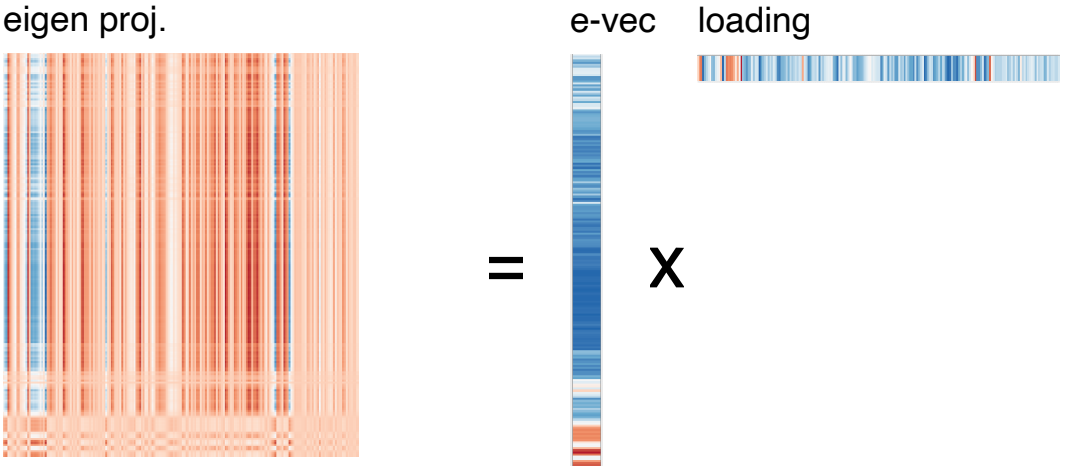
- Orthogonal projection of data into a lower-dimensional **[principal]** sub-space,
- such that the total **variation of the projected data** is maximized

Let's see how much the first eigenvector can explain

Top 200 genes

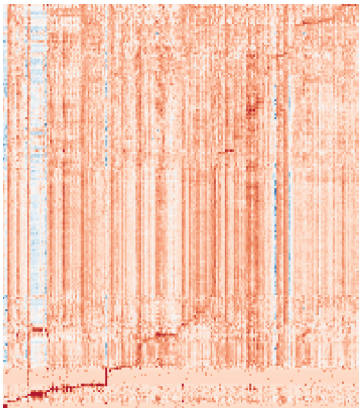


Let's see how much the first eigenvector can explain

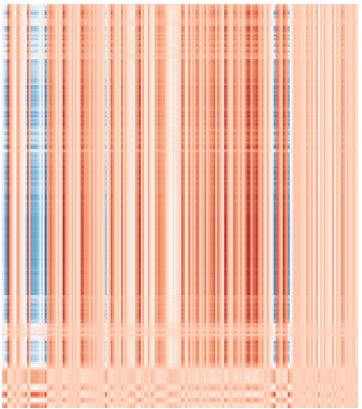


Let's see how much the first eigenvector can explain

Top 200 genes

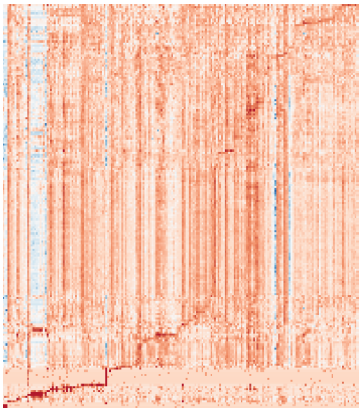


13%



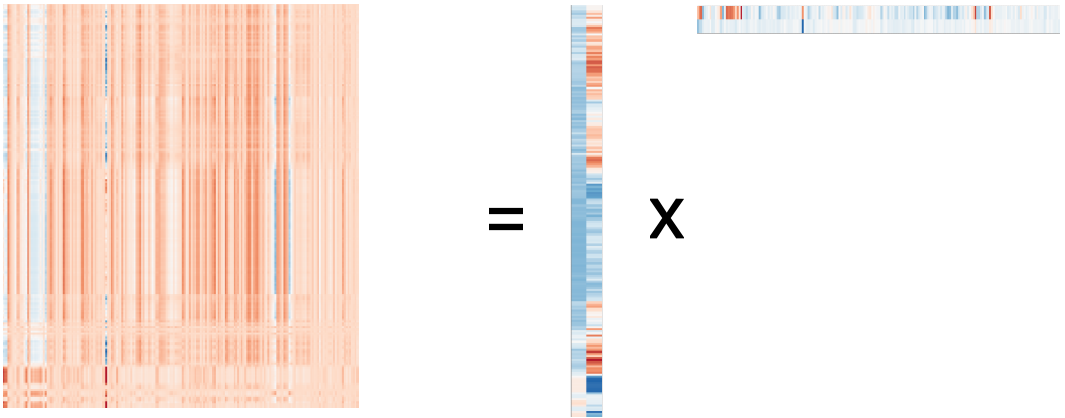
Top two eigen-vectors

Top 200 genes



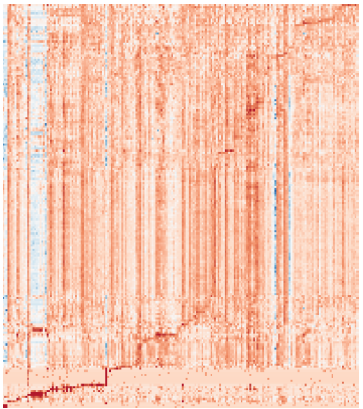
Top two eigen-vectors

eigen proj. e-vec loading

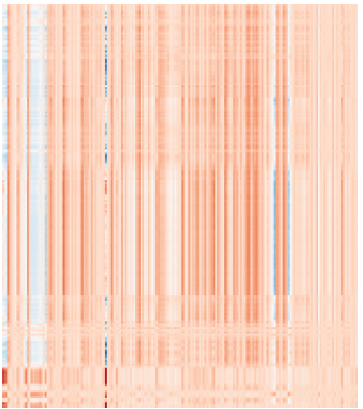


Top two eigen-vectors

Top 200 genes

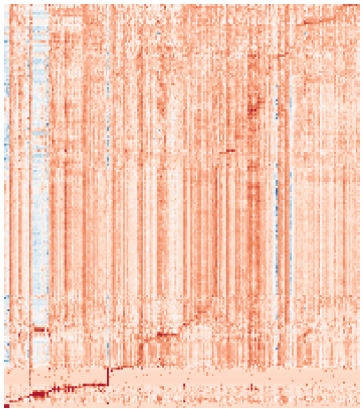


21%



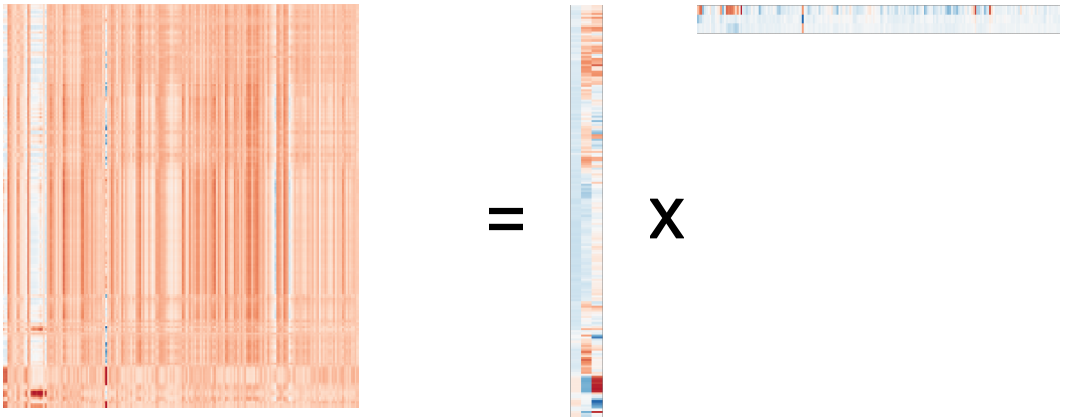
Top three eigen-vectors

Top 200 genes and 200



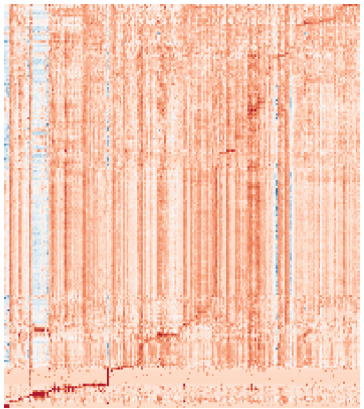
Top three eigen-vectors

eigen proj. e-vec loading

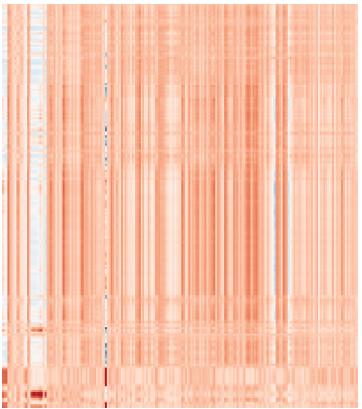


Top three eigen-vectors

Top 200 genes and 200



25%



What is Principal Component Analysis?

(a more mathematical definition)

PCA: Consider this multivariate linear model

$$\begin{pmatrix} X_1 \\ X_2 \\ \vdots \\ X_p \end{pmatrix} = \begin{pmatrix} U_{11} & \cdots & U_{1k} \\ U_{21} & \cdots & U_{2k} \\ \vdots & \vdots & \vdots \\ U_{p1} & \cdots & U_{pk} \end{pmatrix} \begin{pmatrix} V_1 \\ \vdots \\ V_k \end{pmatrix} + \begin{pmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_p \end{pmatrix}$$

or

$\mathbf{x} = U\mathbf{v} + \epsilon$

- If we **knew** U, we would be able to solve weights V.
- And vice versa...

PCA: Consider this multivariate linear model

For many columns, $X = UV + E$, or

$$\begin{pmatrix} X_{11} & \cdots & X_{1n} \\ X_{21} & \cdots & X_{2n} \\ \vdots & & \vdots \\ X_{p1} & \cdots & X_{pn} \end{pmatrix} = \begin{pmatrix} U_{11} & \cdots & U_{1k} \\ U_{21} & \cdots & U_{2k} \\ \vdots & \vdots & \vdots \\ U_{p1} & \cdots & U_{pk} \end{pmatrix} \begin{pmatrix} V_{11} & \cdots & V_{1n} \\ \vdots & & \vdots \\ V_{k1} & \cdots & V_{kn} \end{pmatrix} + \cdots$$

- If we **knew** U, we would be able to solve weights V.
- And vice versa...

PCA: What is a projection matrix?

Suppose we knew V. For each gene (row) g, we solve the following regression.

$$\mathbf{x}_g^\top \sim V\mathbf{u}_g^\top + \epsilon I$$

Least square solution:

$$\min_{\mathbf{u}_g} \|\mathbf{x}_g^\top - V\mathbf{u}_g^\top\|$$

PCA: What is a projection matrix?

Suppose we knew V. For each gene (row) g, we solve the following regression.

$$\begin{aligned} \mathbf{x}_g^\top &\sim V\mathbf{u}_g^\top + \epsilon I \\ \begin{pmatrix} X_{g1} \\ \vdots \\ X_{gn} \end{pmatrix} &\sim \begin{pmatrix} V_{11} & \cdots & V_{1k} \\ \vdots & \vdots & \vdots \\ V_{n1} & \cdots & V_{nk} \end{pmatrix} \begin{pmatrix} U_{g1} \\ \vdots \\ U_{gk} \end{pmatrix} + \begin{pmatrix} \epsilon \\ \vdots \\ \epsilon \end{pmatrix} \end{aligned}$$

Least square solution:

$$\min_{\mathbf{u}_g} \|\mathbf{x}_g^\top - V\mathbf{u}_g^\top\|$$

PCA: What is a projection matrix?

Suppose we knew V. For each gene (row) g, we solve the following regression.

$$\begin{aligned} \mathbf{x}_g^\top &\sim V\mathbf{u}_g^\top + \epsilon I \\ \begin{pmatrix} X_{g1} \\ \vdots \\ X_{gn} \end{pmatrix} &\sim \begin{pmatrix} V_{11} & \cdots & V_{1k} \\ \vdots & \vdots & \vdots \\ V_{n1} & \cdots & V_{nk} \end{pmatrix} \begin{pmatrix} U_{g1} \\ \vdots \\ U_{gk} \end{pmatrix} + \begin{pmatrix} \epsilon \\ \vdots \\ \epsilon \end{pmatrix} \end{aligned}$$

Least square solution:

$$\hat{\mathbf{u}}_g^\top = (V^\top V)^{-1} V^\top \mathbf{x}_g^\top$$

PCA: What is a projection matrix?

Plugging

$$\hat{\mathbf{u}}_g^\top = (V^\top V)^{-1} V^\top \mathbf{x}_g^\top$$

into the following prediction model:

$$\hat{\mathbf{x}}_g^\top = V\hat{\mathbf{u}}_g^\top$$

PCA: What is a projection matrix?

Plugging

$$\hat{\mathbf{u}}_g^\top = (V^\top V)^{-1} V^\top \mathbf{x}_g^\top$$

into the following prediction model:

$$\begin{aligned} \hat{\mathbf{x}}_g^\top &= V\hat{\mathbf{u}}_g^\top \\ &= V(V^\top V)^{-1} V^\top \mathbf{x}_g^\top \end{aligned}$$

PCA: What is a projection matrix?

Plugging

$$\hat{\mathbf{u}}_g^\top = (V^\top V)^{-1} V^\top \mathbf{x}_g^\top$$

into the following prediction model:

$$\begin{aligned} \hat{\mathbf{x}}_g^\top &= V \hat{\mathbf{u}}_g^\top \\ \hat{\mathbf{x}}_g &= \mathbf{x}_g \underbrace{V(V^\top V)^{-1} V^\top}_{n \times n \text{ projection matrix}} \end{aligned}$$

PCA: Projection can be done on the other side

A multivariate regression model for sample (column) j:

$$\mathbf{x}_j \sim U\mathbf{v}_j + \epsilon$$

The least-square solution for the V:

$$\hat{\mathbf{v}}_j = (U^\top U)^{-1} U^\top \mathbf{x}_j$$

Then we have

$$\hat{\mathbf{x}}_j = \underbrace{U(U^\top U)^{-1} U^\top}_{p \times p \text{ projection matrix}} \mathbf{x}_j$$

PCA: maximizing total variance of the projected

Constrained optimization

Given the rank-1 factorization

$$\hat{X} = \mathbf{u}\mathbf{v}^\top$$

and the least square solution $\hat{\mathbf{v}}$

$$\hat{\mathbf{v}}^\top = \frac{\mathbf{u}^\top}{\mathbf{u}^\top\mathbf{u}} X$$

We want to maximize the variance of this projected vector $\mathbf{v}$

$$\hat{\mathbf{v}}^\top\hat{\mathbf{v}} = \frac{\mathbf{u}^\top}{\mathbf{u}^\top\mathbf{u}} XX^\top \frac{\mathbf{u}}{\mathbf{u}^\top\mathbf{u}} = \mathbf{u}^\top XX^\top\mathbf{u}$$

where we assume $\mathbf{u}$ is a unit vector.

PCA is an eigen value problem

PCA

Letting the covariance matrix $\hat{\Sigma} = XX^\top/(p-1)$, we want to find a unit vector $\mathbf{v}$ by

$$\max \mathbf{v}^\top \hat{\Sigma} \mathbf{v}$$

subject to $\mathbf{v}^\top \mathbf{v} = 1$.

Eigen value problem

Given the covariance matrix $\hat{\Sigma}$, we can resolve an eigen-value λ and the corresponding eigen-vector $\mathbf{v}$ such that

$$\hat{\Sigma}\mathbf{v} = \lambda \mathbf{v}$$

SVD: another equivalent method for PCA

Singular Value Decomposition

SVD identifies three matrices of X:

$$X = UDV^\top$$

where both U and V vectors are orthonormal, namely,

- $U^\top U = I$, $\mathbf{u}_k^\top \mathbf{u}_k = 1$ for all k,
- $V^\top V = I$, $\mathbf{v}_k^\top \mathbf{v}_k = 1$ for all k.

Covariance by SVD

Covariance across the columns (samples)

$$X^\top X/p = VD^2V^\top/p$$

Covariance across the rows (genes)

$$XX^\top/n = UD^2U^\top/n$$

Remark: The covariance formulas above assume data is centered (mean-subtracted).

SVD: another equivalent method for PCA

We can confirm the equivalent relations by multiplying singular vectors to the covariance matrix:

$$\underbrace{\frac{\left(X^\top X\right)}{p-1}}_{\text{sample covariance}} \mathbf{v}_1 \quad \propto \quad (\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k) \begin{pmatrix} D_1^2 & 0 & \ldots & \ldots \\ 0 & D_2^2 & 0 & \ldots \\ 0 & \ldots & \ddots & 0 \\ 0 & \ldots & 0 & D_k^2 \end{pmatrix} \begin{pmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \\ \vdots \\ \mathbf{v}_k^\top \end{pmatrix} \mathbf{v}_1$$

SVD: another equivalent method for PCA

We can confirm the equivalent relations by multiplying singular vectors to the covariance matrix:

$$\underbrace{\frac{\left(X^\top X\right)}{p-1}}_{\text{sample covariance}} \mathbf{v}_1 \propto (\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k) \begin{pmatrix} D_1^2 & 0 & \ldots & \ldots \\ 0 & D_2^2 & 0 & \ldots \\ 0 & \ldots & \ddots & 0 \\ 0 & \ldots & 0 & D_k^2 \end{pmatrix} \begin{pmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \\ \vdots \\ \mathbf{v}_k^\top \end{pmatrix} \mathbf{v}_1$$

$$\propto (\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k) \begin{pmatrix} D_1^2 & 0 & \ldots & \ldots \\ 0 & D_2^2 & 0 & \ldots \\ 0 & \ldots & \ddots & 0 \\ 0 & \ldots & 0 & D_k^2 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

SVD: another equivalent method for PCA

We can confirm the equivalent relations by multiplying singular vectors to the covariance matrix:

$$\underbrace{\frac{\left(X^\top X\right)}{p-1}}_{\text{sample covariance}} \mathbf{v}_1 \propto (\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k) \begin{pmatrix} D_1^2 & 0 & \ldots & \ldots \\ 0 & D_2^2 & 0 & \ldots \\ 0 & \ldots & \ddots & 0 \\ 0 & \ldots & 0 & D_k^2 \end{pmatrix} \begin{pmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \\ \vdots \\ \mathbf{v}_k^\top \end{pmatrix} \mathbf{v}_1$$

$$\propto (\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k) \begin{pmatrix} D_1^2 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

SVD: another equivalent method for PCA

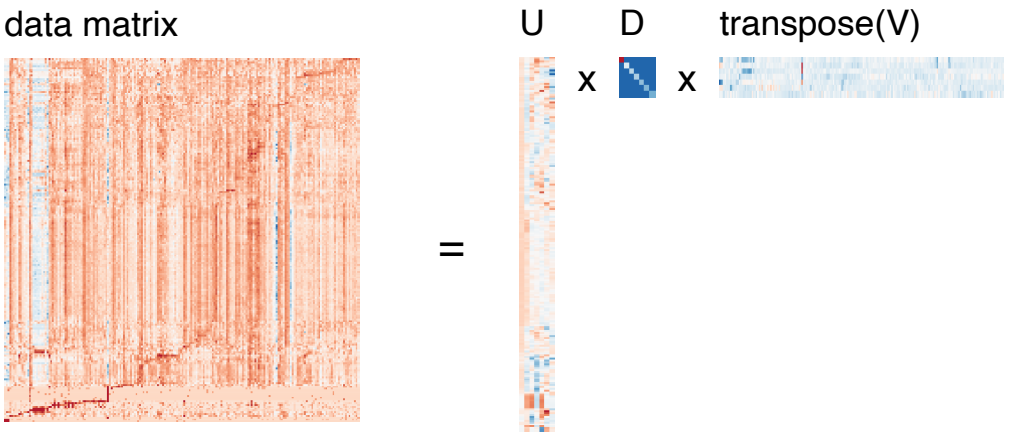
We can confirm the equivalent relations by multiplying singular vectors to the covariance matrix:

$$\underbrace{\frac{\left(X^\top X\right)}{p-1}}_{\text{sample covariance}} \mathbf{v}_1 \quad \propto \quad (\mathbf{v}_1, \mathbf{v}_2, \ldots, \mathbf{v}_k) \begin{pmatrix} D_1^2 & 0 & \ldots & \ldots \\ 0 & D_2^2 & 0 & \ldots \\ 0 & \ldots & \ddots & 0 \\ 0 & \ldots & 0 & D_k^2 \end{pmatrix} \begin{pmatrix} \mathbf{v}_1^\top \\ \mathbf{v}_2^\top \\ \vdots \\ \mathbf{v}_k^\top \end{pmatrix} \mathbf{v}_1$$

$$= \underbrace{\frac{D_1^2}{p-1}}_{\text{eigenvalue}} \underbrace{\mathbf{v}_1}_{\text{eigenvector}}$$

Run SVD to find principal components

```
svd.out <- rsvd::rsvd(x.sorted, k = 7)
U <- svd.out$u; D <- diag(svd.out$d); V <- svd.out$v
```



Summary: PCA

- Matrix factorization = alternating linear regressions
- PCA is a matrix factorization
- PCA can be found by Eigen value/vector problem
- PCA can be found by Singular Value Decomposition

Generalized Linear Model PCA to model count data

- scRNA-seq data are **counts** (discrete, non-negative)

$$\log(\mu_{ij}) = (UV^\top)_{ij}$$

$$Y_{ij} \sim \text{Poisson}(\mu_{ij})$$

or

$$Y_{ij} \sim \text{NB}(\mu_{ij}, \phi)$$

```r
library(glmpca)

## Run GLM-PCA on count matrix
res <- glmpca(X, L = 5, fam = "nb")

factors <- res$factors
loadings <- res$loadings
```

GLM-PCA vs standard PCA

When to use GLM-PCA?

- Working with **raw counts** (not normalized)
- Want to avoid log-transformation artifacts
- More computationally intensive than standard PCA

Reference: (Townes et al. 2019)

Today's lecture

1. What is Principal Component Analysis?
2. Non-negative Matrix Factorization
3. How can we use factorization results?

Why Non-negative Matrix Factorization (NMF)?

PCA (SVD) can have negative values:

E.g., For some gene g: We can have

$$\begin{aligned} U_{g1} + U_{g2} &= 0 \\ \text{if } U_{g1} &= -U_{g2} < 0 \\ \text{or } U_{g1} &= U_{g2} = 0 \\ &\vdots \end{aligned}$$

NMF constraint:

$$X \approx UV$$

where $U, V \geq 0$

Classic NMF in Genomics (metagenes)

Brunet et al. (2004):

- Discovers **"metagenes"** (coherent patterns)
- Non-negative gene/sample combinations
- Only additive combinations (vs. PCA's +/- components)

- Metagenes = pathways/processes
- Genes are "on" or "off" (not negative)
- Sparse representation
- Natural for expression data

Applications:

- Cancer subtype discovery
- Gene expression pattern analysis
- Identifying co-regulated gene modules

Brunet et al. (2004)

NMF for single-cell RNA-seq

Why NMF is particularly useful for scRNA-seq:

- Gene expression is naturally non-negative (counts)
- Identifies **gene programs** (co-expressed gene sets)
- Discovers **cell states** and gradients
- Integrates datasets across batches/conditions
- More interpretable than PCA for biological processes

Two popular tools: `rliger` and `fasttopic`

`rliger`: Integrative NMF for single-cell data

- Joint NMF across multiple datasets
- Shared factors (W) capture common biology
- Dataset-specific factors (V_i) capture batch effects
- Quantile normalization aligns factor loadings

$$X^{(d)} \approx H^{(d)}(W + V^{(d)})^\top$$

$H^{(d)}$: cell factor loadings for dataset d
W: shared gene factors, $V^{(d)}$: dataset-specific

Welch et al. (2019); Gao et al. (2021); Kriebel and Welch (2022)

rliger: Running integrative NMF

```r
library(rliger)

## Load three PDAC tissue samples
tissues <- c("PDAC_TISSUE_1", "PDAC_TISSUE_2", "PDAC_TISSUE_3")

data.list <- lapply(tissues, function(tissue) {
  data <- Read10X(data.dir = paste0("../data/GSE155698_PDAC_Steele2020/", tissue, "/"))
  ## Remove MT-genes
  mt.genes <- grep("^MT-", rownames(data), value = TRUE)
  data[!rownames(data) %in% mt.genes, ]
})
names(data.list) <- tissues
```

`rliger`: Create object and preprocess

```
## Create liger object with three datasets
liger_obj <- createLiger(data.list)

## Normalize and select variable genes
liger_obj <- normalize(liger_obj)
liger_obj <- selectGenes(liger_obj)
liger_obj <- scaleNotCenter(liger_obj)
```

rliger: Run integrative NMF

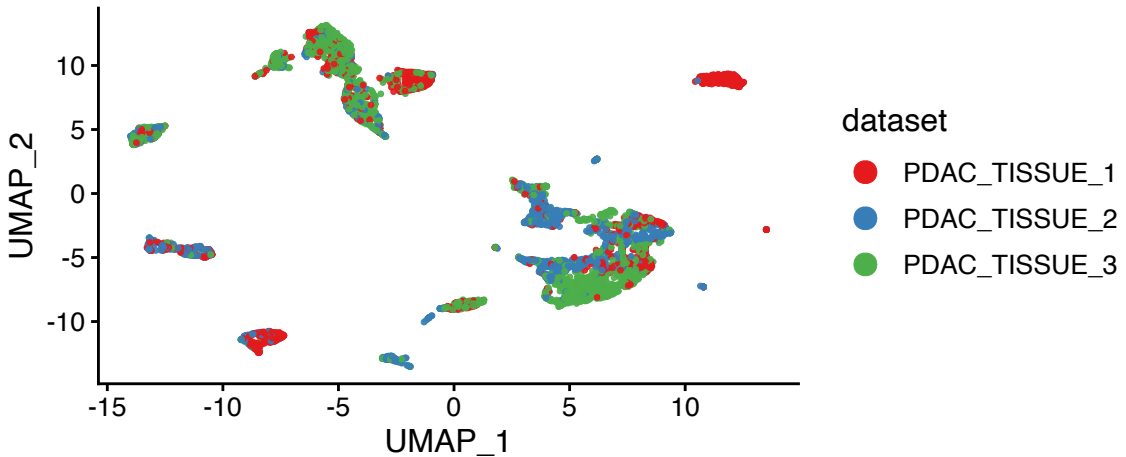
```
## Run integrative NMF
liger_obj <- optimizeALS(liger_obj, k = 20)
```

`rliger`: Align datasets and cluster

```r
## Quantile normalization for alignment
liger_obj <- quantile_norm(liger_obj)

## Clustering on aligned factors
liger_obj <- louvainCluster(liger_obj, resolution = 0.4)

## UMAP visualization
liger_obj <- runUMAP(liger_obj, n_neighbors = 30)
```

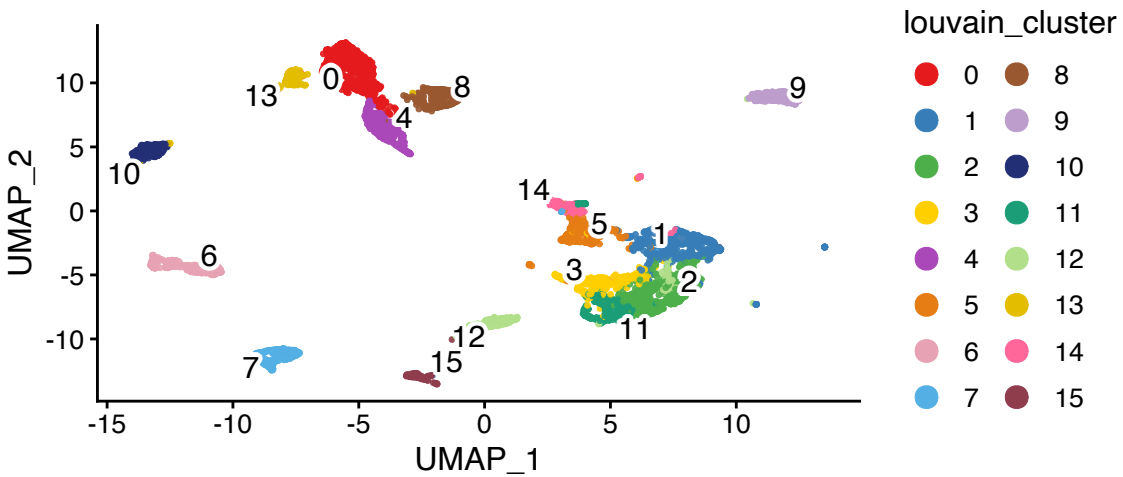
rliger: UMAP by dataset

```
## UMAP colored by dataset
plotDatasetDimRed(liger_obj)
```



rliger: UMAP by cluster

```
## UMAP colored by cluster
plotClusterDimRed(liger_obj)
```



`fastTopics`: Topic modeling for scRNA-seq

- Topic model: each cell F = **mixture of topics**
- each topic L = distribution over genes

$$Y_{gj} = \sum_k L_{gk} F_{kj}$$

Carbonetto et al. (2021); Carbonetto et al. (2023)

```
library(fastTopics)

## Use the same PDAC tissue 1 count matrix (cells x genes)
## Select top 5000 most variable genes
gene_var <- apply(X, 1, var)
top_genes <- names(sort(gene_var, decreasing = TRUE))[1:5000]
counts <- t(X[top_genes, ])
dim(counts)
```

fastTopics: Fit topic model

```
## Fit topic model with k=10 topics
fit <- fit_topic_model(counts, k = 10)
```

fastTopics: Topic proportions per cell

```r
## L matrix: cells x topics (topic proportions)
head(round(fit$L, 3))
```

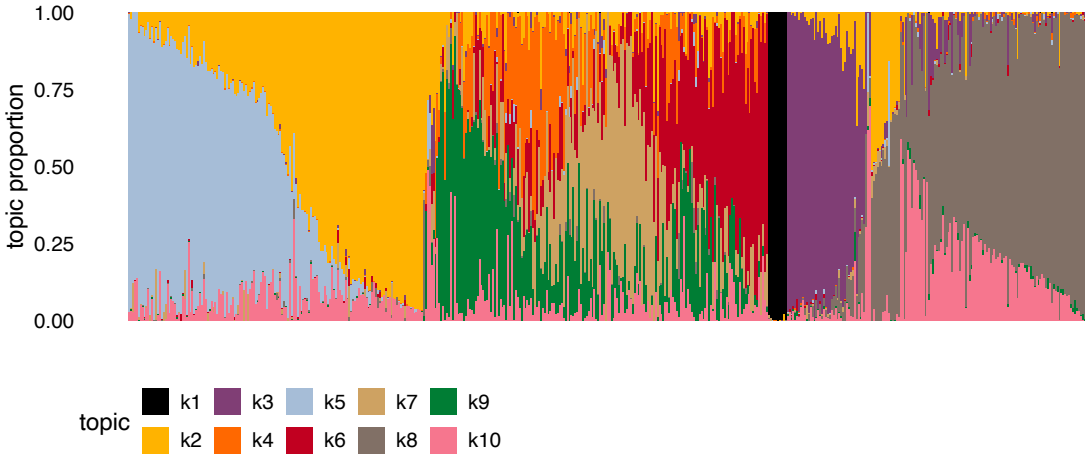
```
##                     k1    k2    k3    k4    k5   k6    k7    k8    k9   k10
## AAACGAAAGTGGAAAG-1   0 0.000 0.000 0.006 0.000 0.00 0.000 0.407 0.000 0.586
## AAACGAAGTAGGGTAC-1   0 0.000 0.000 0.002 0.000 0.55 0.000 0.004 0.292 0.152
## AAACGAAGTCATAGTC-1   0 0.028 0.000 0.523 0.000 0.00 0.161 0.000 0.225 0.062
## AAAGAACCATTAAAGG-1   0 0.114 0.761 0.000 0.011 0.00 0.000 0.050 0.023 0.040
## AAAGGATTCGGCTTGG-1   0 0.233 0.000 0.003 0.635 0.00 0.000 0.000 0.000 0.128
## AAAGGGCAGTAGCAAT-1   0 0.140 0.650 0.001 0.000 0.00 0.009 0.182 0.005 0.014
```

fastTopics: Top genes per topic

```r
## F matrix: genes x topics (gene weights)
top_genes <- apply(fit$F, 2, function(x) {
  names(sort(x, decreasing = TRUE)[1:10])
})
top_genes[, 1:3]
```

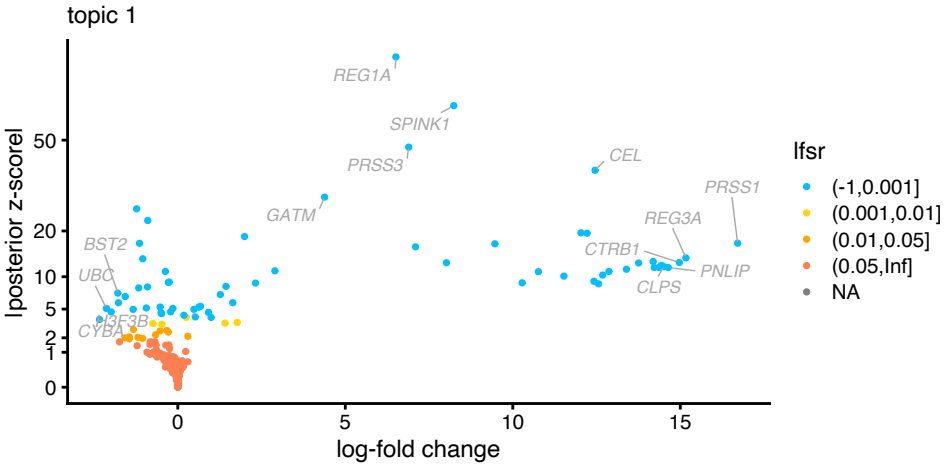
```
##        k1        k2        k3
##  [1,] "PRSS1"   "MALAT1"  "FTL"
##  [2,] "REG3A"   "RPS27"   "SPP1"
##  [3,] "CTRB1"   "RPL41"   "FTH1"
##  [4,] "PNLIP"   "RPL10"   "B2M"
##  [5,] "REG1A"   "RPL13"   "CTSD"
##  [6,] "CPB1"    "EEF1A1"  "TMSB4X"
##  [7,] "CPA1"    "TPT1"    "APOE"
##  [8,] "CLPS"    "B2M"     "HLA-B"
##  [9,] "CTRB2"   "RPS18"   "CTSB"
## [10,] "CELA3A"  "RPS12"   "ACTB"
```

fastTopics: Structure plot



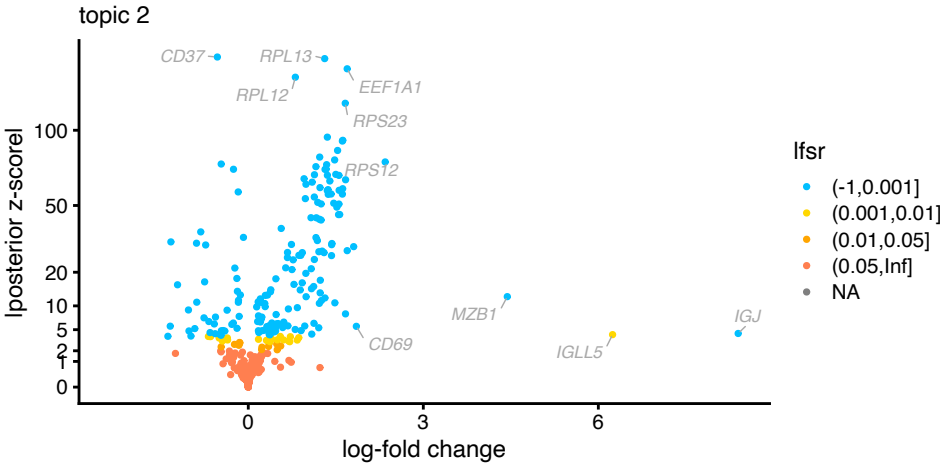
`fastTopics`: Volcano plot — genes driving each topic

Topic 1: PRSS1, REG3A, CTRB1 — pancreatic acinar cells



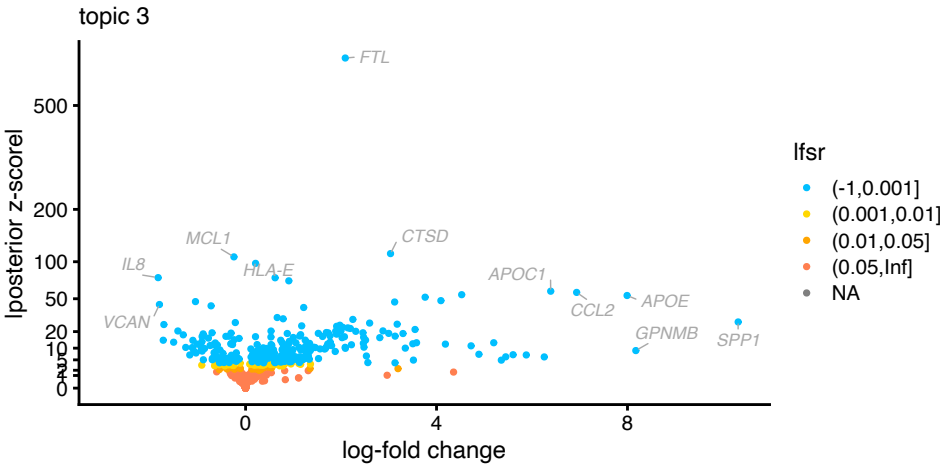
`fastTopics`: Volcano plot — genes driving each topic

Topic 2: MALAT1, RPL41, RPL10 — housekeeping/ribosomal



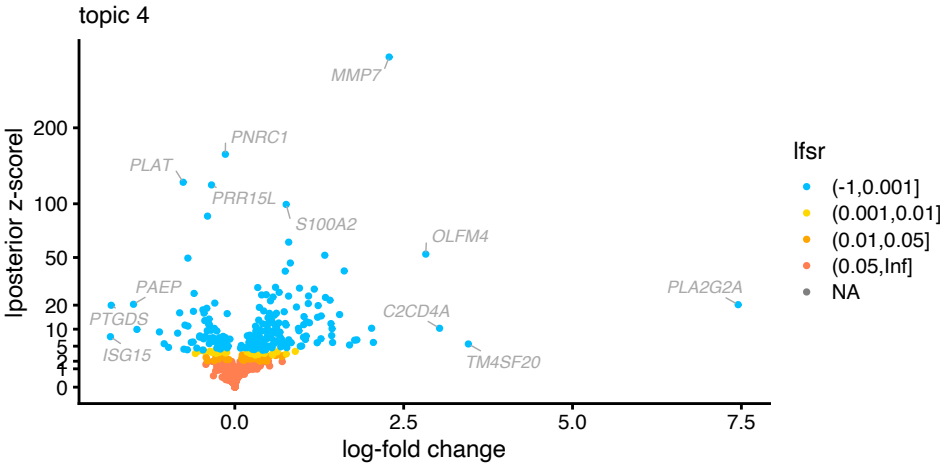
`fastTopics`: Volcano plot — genes driving each topic

Topic 3: FTL, SPP1, CTSD — macrophage/myeloid markers



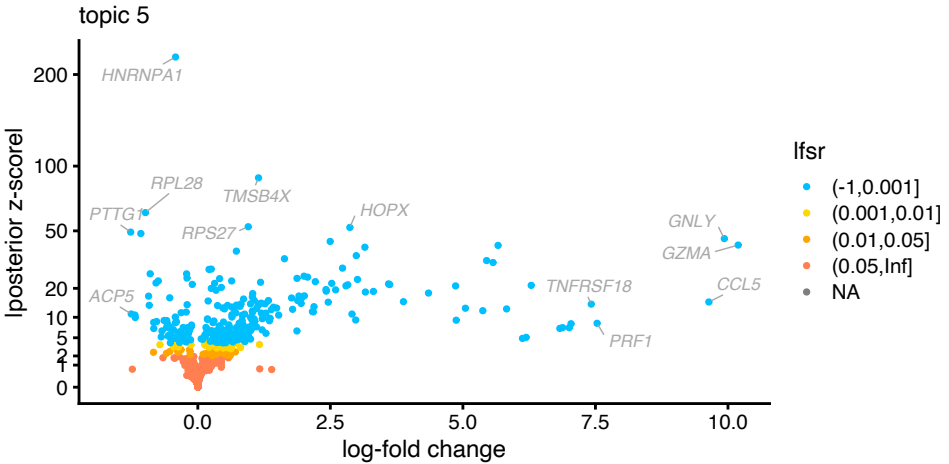
`fastTopics`: Volcano plot — genes driving each topic

Topic 4: RARRES1, PLA2G2A, RPL18A — ductal/regenerating



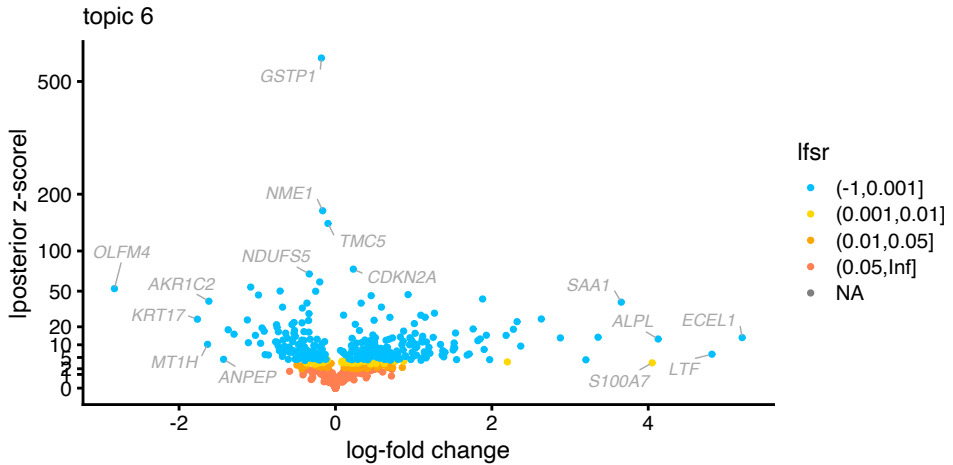
`fastTopics`: Volcano plot — genes driving each topic

Topic 5: TMSB4X, GNLY, GZMA — cytotoxic T/NK cells



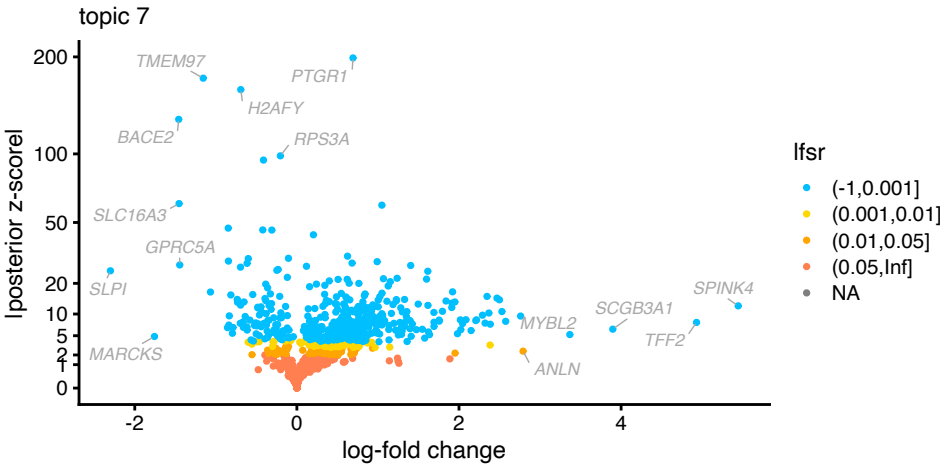
`fastTopics`: Volcano plot — genes driving each topic

Topic 6: RARRES1, FTH1, HLA-B — ductal/epithelial



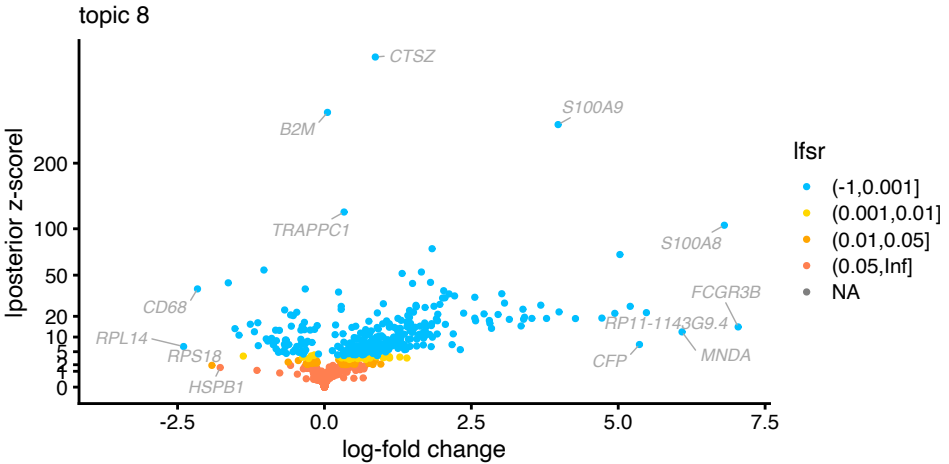
`fastTopics`: Volcano plot — genes driving each topic

Topic 7: S100A6, RPLP1, RPS19 — epithelial



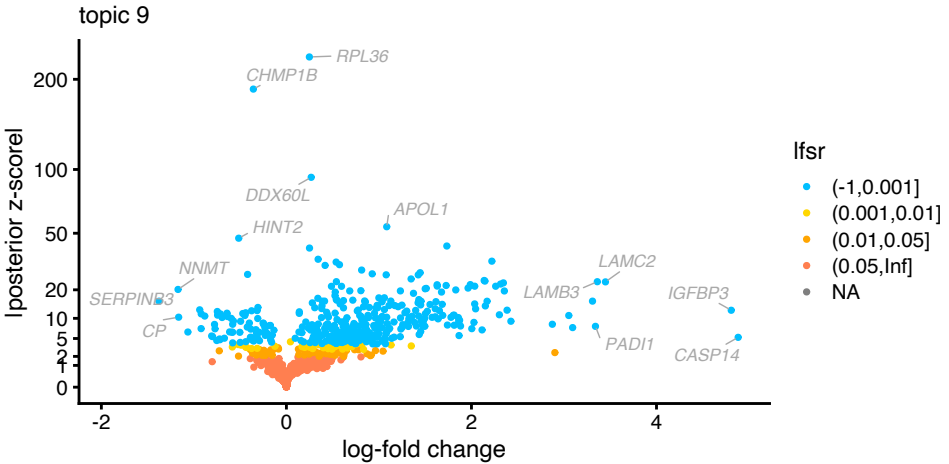
fastTopics: Volcano plot — genes driving each topic

Topic 8: S100A9, S100A8, LYZ — neutrophils/monocytes



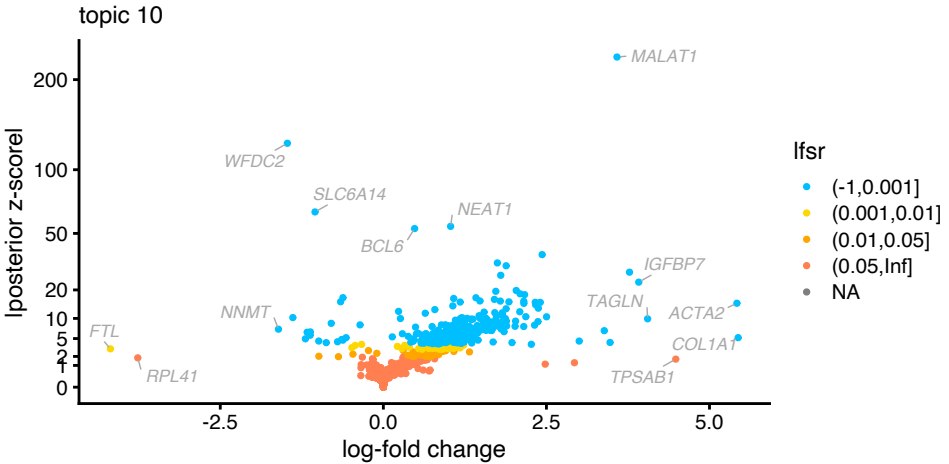
`fastTopics`: Volcano plot — genes driving each topic

Topic 9: S100A6, FTH1, TMSB10 — proliferating



`fastTopics`: Volcano plot — genes driving each topic

Topic 10: MALAT1, NEAT1, HLA-A — antigen presentation/stress



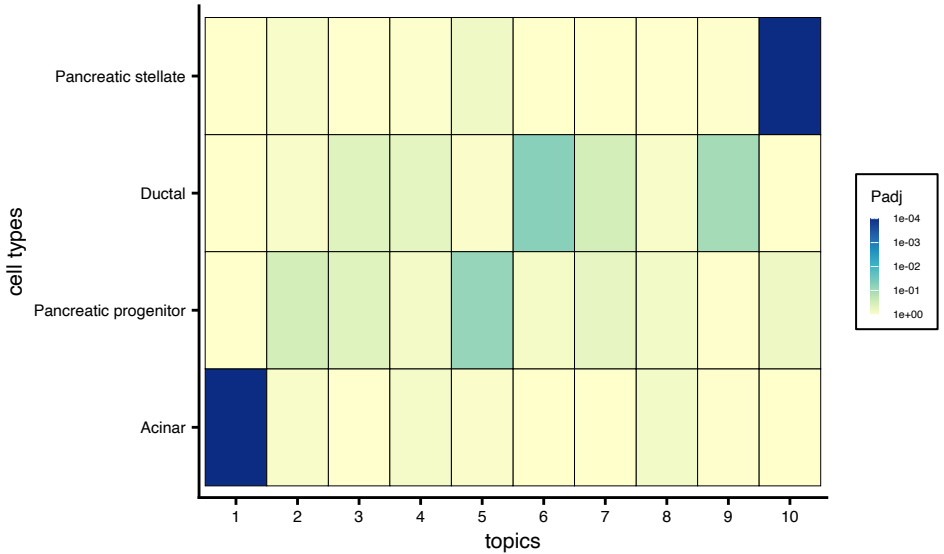
Today's lecture

1. What is Principal Component Analysis?
2. Non-negative Matrix Factorization
3. How can we use factorization results?

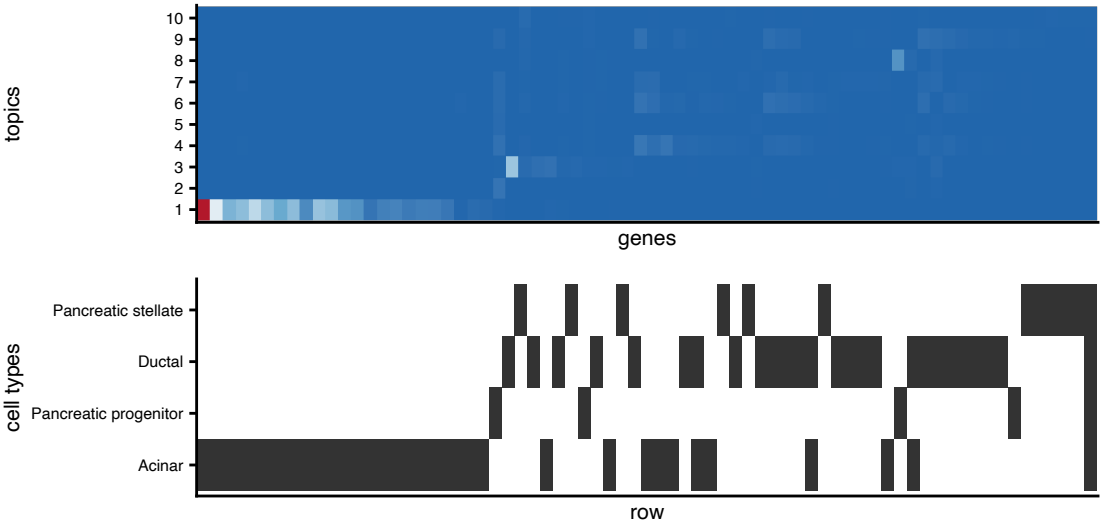
Gene set enrichment analysis with PanglaoDB

```r
library(fgsea)

## Read PanglaoDB cell-type marker gene sets
panglao.db <- read.panglao("Pancreas")

## Prepare gene weights from fastTopics F matrix
## Each column = topic, rows = genes
.beta <- apply(fit$F, 2, scale)
rownames(.beta) <- colnames(counts)

## Sort genes by topic assignment
.genes <- unique(unlist(panglao.db))
.dt <- sort.beta(.beta, genes.selected = .genes)

## Run fgsea per topic
.gsea <- run.beta.gsea(.dt, panglao.db)
```

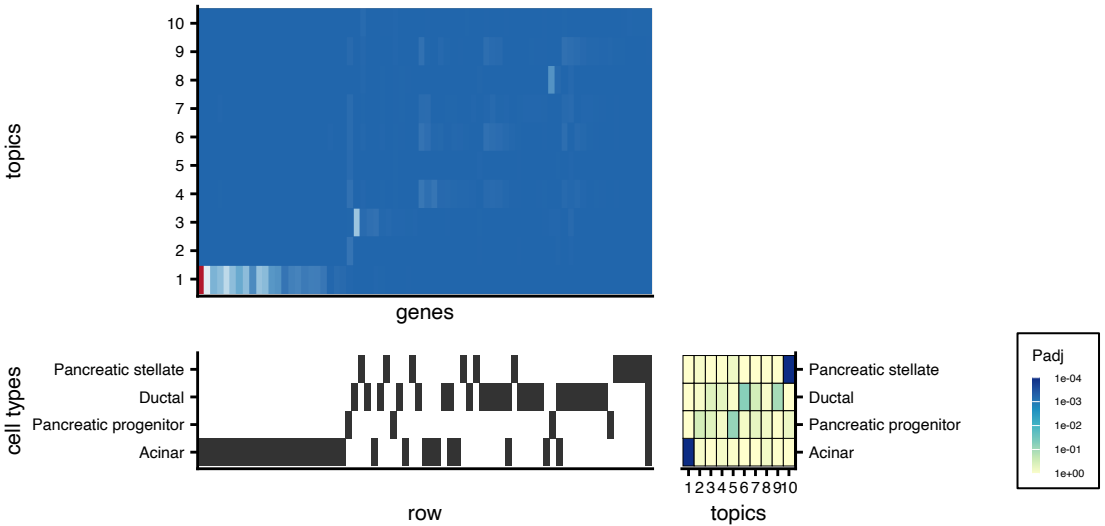
GSEA results: which topics correspond to cell types?



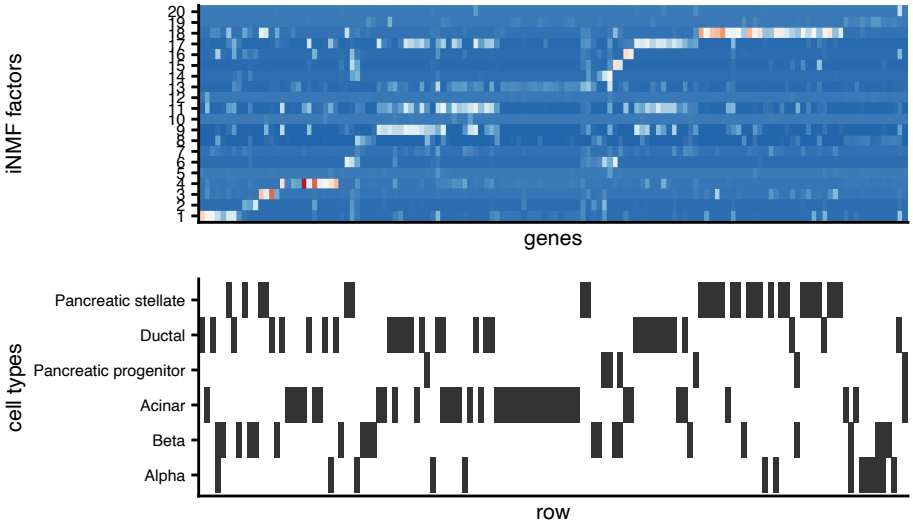
GSEA: Gene weights across topics



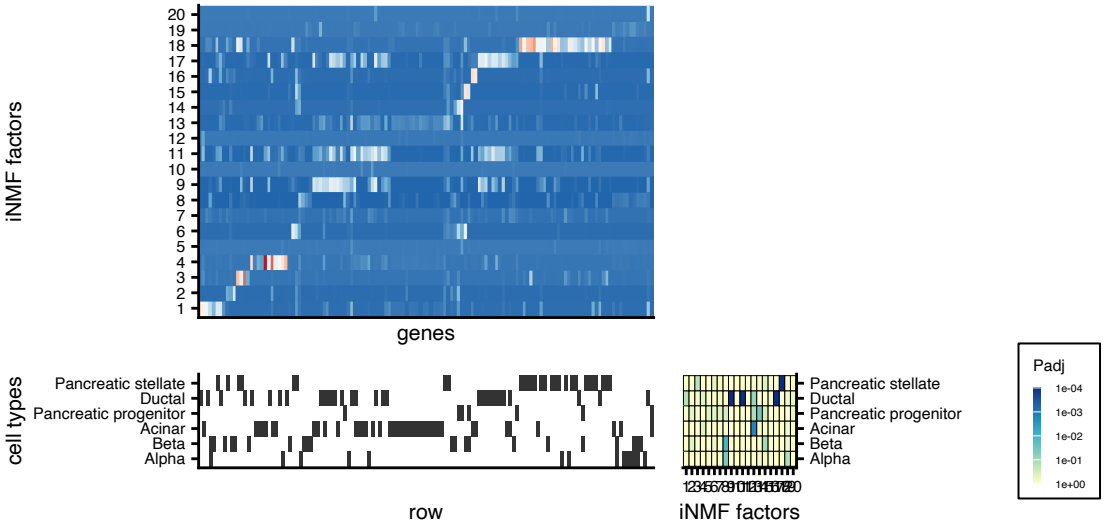
GSEA: Gene weights across topics



`rliger`: GSEA on shared gene loadings (W matrix)



`rliger`: GSEA on shared gene loadings (W matrix)



Wrap-up

- **PCA/SVD**: Find orthogonal axes of maximum variance; fast, good for QC
- **NMF**: Non-negative, parts-based decomposition; interpretable gene programs
- `rliger`: Integrative NMF across multiple datasets (batch correction)
- `fastTopics`: Scalable topic model; cells as mixtures of gene programs
- **GSEA on factors**: Link learned topics to known cell types (e.g., PanglaoDB + `fgsea`)

Reference

Brunet, Jean-Philippe, Pablo Tamayo, Todd R Golub, and Jill P Mesirov. 2004. "Metagenes and Molecular Pattern Discovery Using Matrix Factorization." *Proc. Natl. Acad. Sci. U. S. A.* 101 (12): 4164–69.

Carbonetto, Peter, Kaixuan Luo, Abhishek Sarkar, et al. 2023. "GoM DE: Interpreting Structure in Sequence Count Data with Differential Expression Analysis Allowing for Grades of Membership." *Genome Biol.* 24 (1): 236.

Carbonetto, Peter, Abhishek Sarkar, Zihao Wang, and Matthew Stephens. 2021. "Non-Negative Matrix Factorization Algorithms Greatly Improve Topic Model Fits." *arXiv* 2105.13440.

Reference

Gao, Chao, Jialin Liu, April R Kriebel, et al. 2021. "Iterative Single-Cell Multi-Omic Integration Using Online Learning." *Nat. Biotechnol.* 39 (8): 1000–1007.

Hotelling, Harold. 1933. "Analysis of a Complex of Statistical Variables into Principal Components." *J. Educ. Psychol.* 24 (6): 417–41.

Kharchenko, Peter V. 2021. "The Triumphs and Limitations of Computational Methods for scRNA-Seq." *Nat. Methods* 18 (7): 723–32.

Kriebel, April R, and Joshua D Welch. 2022. "UINMF Performs Mosaic Integration of Single-Cell Multi-Omic Datasets Using Nonnegative Matrix Factorization." *Nat. Commun.* 13 (1): 780.

Reference

Pearson, Karl. 1901. "On Lines and Planes of Closest Fit to Systems of Points in Space." *Philos. Mag.* 2 (11): 559–72.

Townes, F William, Stephanie C Hicks, Martin J Aryee, and Rafael A Irizarry. 2019. "Feature Selection and Dimension Reduction for Single-Cell RNA-Seq Based on a Multinomial Model." *Genome Biol.* 20 (1): 295.

Welch, Joshua D, Velina Kozareva, Ashley Ferreira, Charles Vanderburg, Carly Martin, and Evan Z Macosko. 2019. "Single-Cell Multi-Omic Integration Compares and Contrasts Features of Brain Cell Identity." *Cell* 177 (7): 1873–1887.e17.